Security in Software Applications

Approaches and Tools for code Analysis

SAPIENZA
UNIVERSITÀ DI ROMA

Adapted from David A. Wheeler
https://github.com/david-a-wheeler

Daniele Friolo
friolo@di.uniroma1.it
https://danielefriolo.github.io

Outline

- Types of analysis (static/dynamic/hybrid)
  - Some measurement terminology
- Static analysis
- Dynamic analysis (fuzz testing)
- Hybrid analysis
- Operational
- Fool with a tool
- SWAMP
- Adopting tools

Types of analysis

- Static analysis: Approach for verifying software (including finding defects) without executing software
  - Source code vulnerability scanning tools, code inspections, etc.
- Dynamic analysis: Approach for verifying software (including finding defects) by executing software on specific inputs and checking results ("oracle")
  - Functional testing, fuzz testing, etc.
- Hybrid analysis: Combine above approaches
- Operational: Tools in operational setting
  - Minimize risks, report information back, etc.
  - may be static, dynamic, hybrid; often dynamic

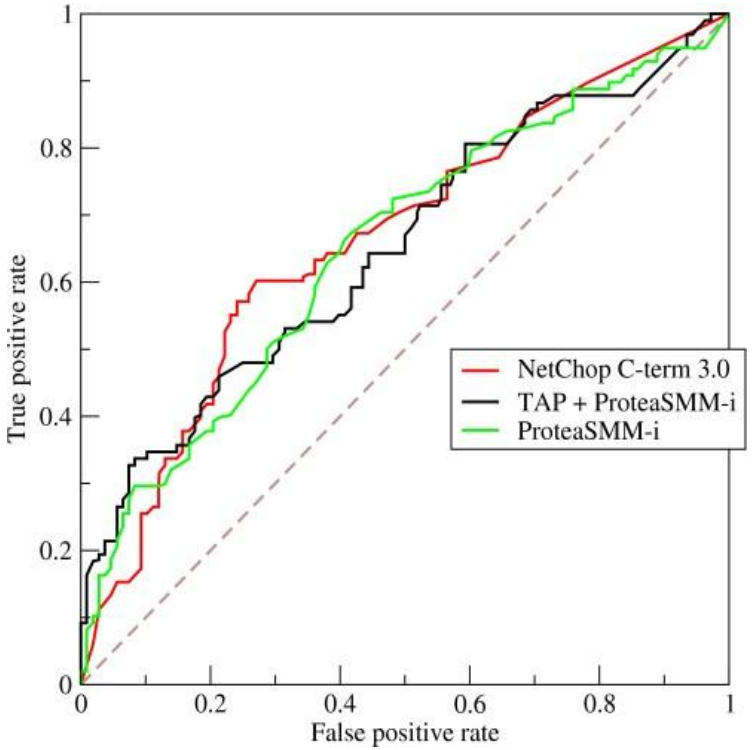
Basic measurement terminology

Analysis/tool report	Report correct	Report incorrect
Reported a defect	**True positive (TP):** Correctly reported a defect	**False positive (FP):** Incorrect, it reported a "defect" that's not a defect ("Type I error")
Did not report a defect (there)	**True negative (TN):** Correctly did not report a (given) defect	**False negative (FN):** Incorrect because it failed to report a defect ("Type II error")

- False positive rate, FPR = #FP/(#TN+#FP) "Probability alert is false"
- True positive rate, TPR = #TP/(#TP + #FN) "% vulnerabilities found" (sensitivity)
- Developers worry about large false positive rate (FPR)
  - "Tool report wasted my time"
- Auditors worry about small or <100% TPR for a given category
  - "Tool missed something important"

Receiver operating characteristic (ROC) curve

- Binary classifiers must generally trade off between FP rates and TP rates
  - To get more reports (larger TP rate), must accept larger FP rate
  - What's more important to you, low FP rate or high TP rate?
- ROC curve (from WW II) graphically illustrates this
- Don't normally know the true values for given tools, but effect is still pronounced
  - Tool developer focus
  - Tool users can configure tool to affect trade-off



Sample ROC curve

[Source: Wikipedia "ROC curve"]

Measurement roll-ups

- Precision (positive predictive value) = #TP/(#TP+#FP)
  - can be seen as a measure of **quality**
- Recall (sensitivity, soundness, find rate) =

  #TP/(#TP+#FN)
  - Can be seen as a measure of **quantity**

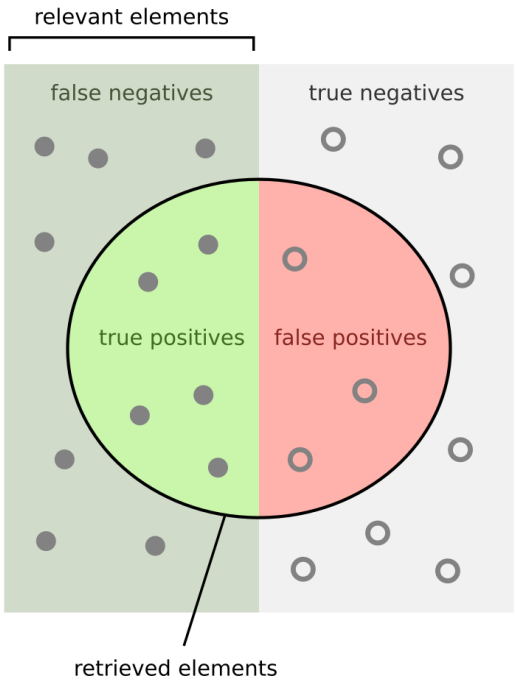
    % vulnerabilities found
- F-score (harmonic mean) =

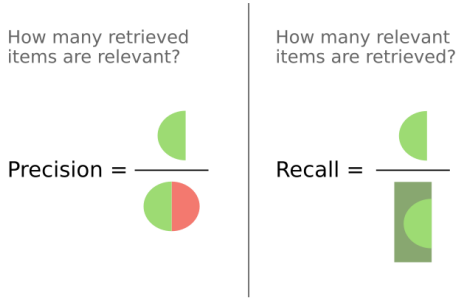
  2 x (Precision x Recall) / (Precision + Recall)
- Discrimination rate =

  #Discriminations / #Test_Pairs
  - Given a pair of tests (one with defect, one without)
  - Discrimination occurs if tool correctly reports flaw (TP) in test with flaw AND does not when there is no flaw (TN)
- All these values 0..1, where higher is better

http://samate.nist.gov/docs/CAS_2011_SA_Tool_Method.pdf

Source: CAS Static Analysis Tool Study - Methodology (Dec 2011)





Source: Wikipedia

Static Analysis

SAPIENZA
UNIVERSITÀ DI ROMA

Source vs. Executables

- Source code pros:
  - Provides much more context; executable-only tools can miss important information
    - Can examine variable names and comments (can be very helpful!)
    - Can *fix* problems found (hard with just executable)
    - Difficult to decompile code
  - Source code cons:
    - Can mislead tools – executable runs, not source (if there's a difference)
    - Often cannot get source for proprietary off-the-shelf programs
      - Can get for open source software
      - Often can get for custom
  - Bytecode is somewhere between

(Some) Static analysis approaches

- Human analysis (including peer reviews)
- Type checkers
- Compiler warnings
- Style checkers / defect finders / quality scanners
- Security analysis:
  - Security weakness analysis - text scanners
  - Security weakness analysis - beyond text scanners
- Property checkers
- Knowledge extraction
- formal methods (separately)

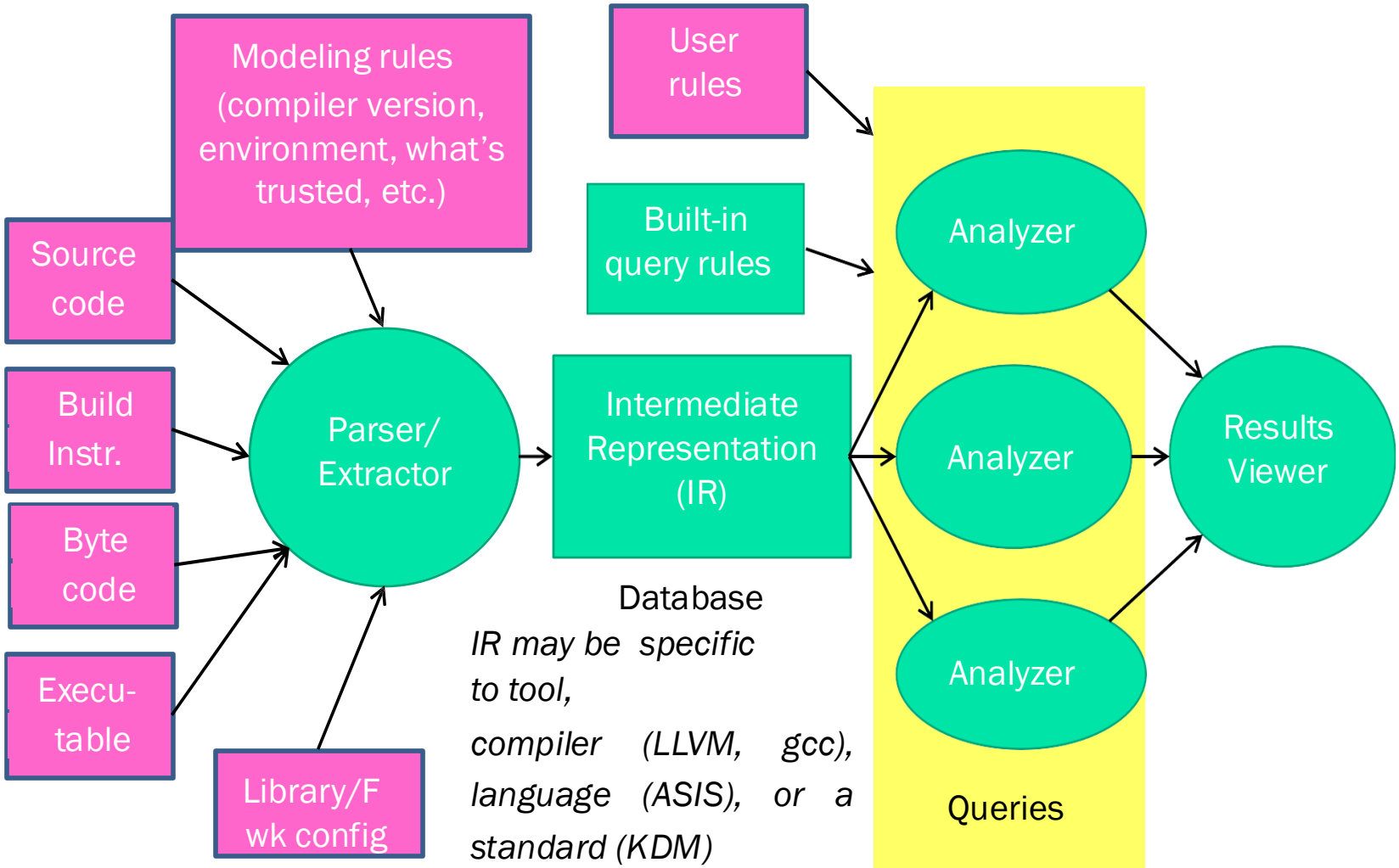
Human (manual) analysis

- Humans are great at discerning context and intent
- Get bored and overwhelmed
- Expensive
  - Especially if analyzing executables
- Can be one person, e.g., “desk-checking”
- Peer reviews
  - Inspections: Special way to use group, defined roles including “reader”; see IEEE standard 1028
- Can focus on specific issues
  - e.g., “Is everything that’s *supposed* be authenticated covered by authentication processes?”

Automated tool limitations

- Tools typically don't "understand":
  - System architecture
  - System mission/goal
  - Technical environment
  - Human environment
- Except for formal methods…
  - Most have significant FP and/or FN rates
- Best when *part* of a process to develop secure software, not as the only mechanism

Automated tool limitations



Type checkers

- Many languages have static type checking built in
  - Some more rigorous than others
  - C/C++ not very strong (and must often work around)
  - Java/C# stronger (interfaces, etc., ease use)
- Can detect some defects before fielding
  - Including some security defects
  - Also really useful in documenting intent
- Work *with* type system – be as narrow as you can
  - Beware diminishing returns

PMD vs. FindBugs

- Both PMD and FindBugs:
  - Focus on “quality” issues, not security
  - Open source software, available no cost
  - Operate on Java
- PMD
  - Works on (Java) source code
  - Examples: Violation of naming conventions, lack of curly braces, misplaced null check, unnecessary constructor, missing break in switch
  - Also provides Cyclomatic complexity
- FindBugs
  - Works on (JVM) bytecode
  - Examples: equals() method fails on subtypes, clone method may return null, reference comparison of Boolean values, impossible cast, 32bit int shifted by an amount not in the range of 0-31, a collection which contains itself, equals method always returns true
- Neither good at finding security issues – not their purpose

Security defect text scanners

- Scan source code using simple grep-like lexer
  - Typically “know” about comments and strings
  - Look for function calls likely to be problematic
- Examples:
  - RATS (Rough Auditing Tool for Security)
    https://github.com/andrew-d/rough-auditing-tool-for-security
  - ITS4
    https://www.synopsys.com/content/dam/synopsys/sig-assets/reports/its4.pdf
  - Flawfinder (D.A.Wheeler)
    https://dwheeler.com/flawfinder/
- Pros:
  - Fast and cheap
  - Can process partial code (including un-compilable code)
- Cons:
  - Lack of context leads to large FN & FP rates
  - Useful primarily for warning of “dangerous” functions

Analysis approach: Examining structure / Method calls

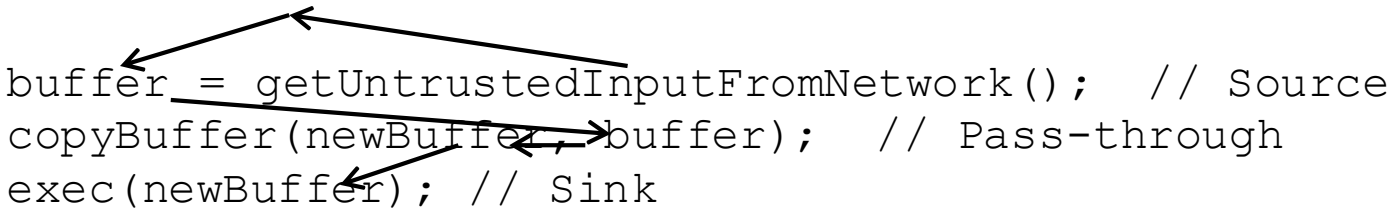
- Warn about calls to `gets()`:

  FunctionCall: function is [`name =="gets"`]

Analysis approach: Taint propagation

- Many tools (static and dynamic) perform “taint propagation”
  - Input from untrusted users (“sources”) considered “tainted”
  - Warn/forbid sending tainted data to certain methods and constructs (“sinks”)
  - Some operations (e.g., checking) may “untaint” data
- Static analysis:
  - Follow data flow from sources through program
  - Determine if tainted data can get to vulnerable “sink”
- Dynamic analysis (e.g., Perl, Ruby):
  - Variables have “taint” value set when input from some sources
  - Certain operations (sinks) forbid direct use of tainted data
    - Counters accidental use of untrusted and unchecked data
    - esp. useful on injection (SQL, command) and buffer overflow

Taint propagation example



- Source rule:
  - Function: `getUntrustedInputFromNetwork()`
  - Postcondition: return value is tainted
- Pass-through rule:
  - Function: `copyBuffer()`
  - Postcondition: If `arg2` tainted, then `arg1` tainted
- Sink rule:
  - Function: `exec()`
  - Precondition: `arg1` must not be tainted

Knowledge Extraction / Program Understanding

- Create view of software automatically for analysis
  - Especially useful for large code bases
  - Visualizes architecture
  - Enables queries, translation to another language
- Examples:
  - Hatha Systems' Knowledge Refinery
  - IBM Rational Asset Analyzer (RAA)
    - Delivering up-to-date knowledge of application assets from the code itself
    - Making the same application insight available to all team members through a web browser interface
    - Taking an inventory of mainframe and distributed application assets
    - Analyzing the impact of a change on mainframe and distributed application assets
    - Improving the reuse of assets and sharing of knowledge throughout the development cycle
  - Relativity MicroFocus (COBOL-focused)

Source/Byte/Binary code security scanners/analyzers

- https://www.nist.gov/itl/ssd/software-quality-group/samate/samate-tool-survey
  - Click on “Source Code Security Analyzers”, “Byte Code Scanners”, and “Binary Code Scanners”
- http://www.dwheeler.com/flawfinder

Dynamic Analysis

SAPIENZA
UNIVERSITÀ DI ROMA

Dynamic analysis' fundamental issue: Cannot test all inputs

- Given trivial program "add two 64-bit integers"
  - Input space = (2^{64}) (2^{64}) = 2^{128} possibilities
- Checking "all inputs" not realistic even in this case
  - Given 4GHz processor & 5 cycles/input (too fast):

    time=2^{128} inputs * (5 cycles/input) * (1 second/(4GHz cycles))

    = 1.35 x 10^{22} years (13.5 zettayears aka sextillion years)
  - Using 1 million 8-core processors doesn't help:

    time=1.7 x 10^{15} years (petayears aka quadrillion years)
- Real programs have far more complex inputs
  - Even a 1% sample impossible in human lifetimes

Functional testing for security

- Use normal testing approaches, but add tests for security requirements
  - Test both “should happen” and “should not happen”
  - Often people forget to test what “should *not* happen”
    - “Can I read/write without being authorized to do so?”
    - “Can I access the system with an invalid certificate?”
- *Branch/statement* coverage tools may warn you of untested paths
- As always, automate and rerun

Web application scanners

- Attempt to go through the various web forms and links
- Send in attack-like and random data
  - Key issues: Input vector (Query string? HTTP body? JSON? XML?), scan barrier, crawl / input vector extraction, which vulnerabilities it detects (and how well)
  - Often build on “fuzzing” techniques (discussed next)
- Shay Chen reviews many in “The Web Application Vulnerability Scanners Benchmark”
  - See Top ten web application vulnerabilities in https://owasp.org/www-project-top-ten/
  - See 2014 discussion in http://sectooladdict.blogspot.com/2014/02/wavsep-web-application-scanner.html
  - See OWASP recommended tools in https://owasp.org/www-community/Vulnerability_Scanning_Tools

Fuzz testing ("fuzzing")

- Testing technique that:
  - Provides (many!) invalid/random values as inputs
  - Monitors program for crashes and other signs of trouble (failing code assertions, appearance of memory leaks)…
    not if the final answer is "correct" (this process is the "oracle")
- Simplifies "oracle" so can create massive data set
- Do not need source, might not even need executable
  - Useful for pen-testers
- Often quickly finds a number of real defects
  - Attackers use it; do not have easy-to-find vulnerabilities
- Can be *very* useful for security, often finds problems
- Typically diminishing rate of return

Fuzz testing variations: Input

- Fuzz testing concept from Barton Miller's 1988 class project Univ. of Wisconsin
  - Project created "fuzzer" to test reliability of command-line Unix programs
  - Repeatedly generated random data for them until crash/hang
  - Later expanded for GUIs, network protocols, etc.
- Approach quickly found a number of defects
- Many tools and approach variations created since

Fuzz testing variations: oracle

- Originally, just “did it crash / hang”?
- Adding program assertions (enabled!) can reveal more
- Test other “should not happen”
  - Ensure files/directories unchanged if shouldn’t be
  - Memory leak (e.g., *valgrind*)
  - Invalid memory access, e.g., using AddressSanitizer (aka ASan) for C/C++/Objective-C to detect buffer overflows and double-frees
  - More intermediate (external) state checking
  - Final state “valid” (! = “correct”)

Fuzz testing: Problems

- Fully random often does not test much
  - e.g., if input has a checksum, fuzz testing ends up primarily checking the checksum algorithm
- Fuzz testing only finds "shallow" problems
  - Special cases ("`if (a == 2) ...`") rare in input space
  - Sequence of rare-probability events by "random" input will typically not be covered by testing
  - Can modify generators to increase probability… but you have to know very specific defect pattern before you find defect
  - In general, only a small amount of program gets covered
- Once defects found by fuzz testing fixed, fuzz testing has a quickly diminishing rate of return
  - Fuzz testing is still a good idea… but not by itself

Hybrid Analysis

SAPIENZA
UNIVERSITÀ DI ROMA

More hybrid approaches

- Concolic testing ("Concolic" = concrete + symbolic)
  - Hybrid software verification technique that interleaves concrete execution (testing on particular inputs) with symbolic execution
  - Can be combined with fuzz testing for better test coverage to detect vulnerabilities
- Sparks, Embleton, Cunningham, Zou 2007 "*Automated Vulnerability Analysis: Leveraging Control Flow for Evolutionary Input Crafting*" http://www.acsac.org/2007/abstracts/22.html
  - Extends black box fuzz testing with genetic algorithm
  - Uses "*dynamic program instrumentation to gather runtime information about each input's progress on the control flow graph, and using this information, we calculate and assign it a '**fitness**' value. Inputs which make more runtime progress on the control flow graph or explore new, previously unexplored regions receive a higher fitness value. Eventually, the inputs achieving the highest fitness are 'mated' (e.g. combined using various operators) to produce a new generation of inputs…. does not require that source code be available*"
- Hybrid approaches are an active research area

More hybrid approaches (2)

- Dao and Shibayama 2011, "*Security sensitive data flow coverage criterion for automatic security testing of web applications*" (ACM) – proposes new coverage measure, "security sensitive data flow coverage":

  "*This criterion aims to show how well test cases cover security sensitive data flows. We conducted an experiment of automatic security testing of real-world web applications to evaluate the effectiveness of our proposed coverage criterion, which is intended to guide test case generation. The experiment results show that security sensitive data flow coverage helps reduce test cost while keeping the effectiveness of vulnerability detection high.*"

Penetration testing (pen testing)

- Pretend to be adversary, try to break in
- Depends on the skills of the pen testers
- Need to set rules-of-engagement (RoE)
  - Problem: RoE often unrealistic
- Really a combination of static and dynamic approaches

Operational

SAPIENZA
UNIVERSITÀ DI ROMA

What about when it's fielded?

- Hook into logging systems
  - Make sure your logging system is flexible and can hook into common logging systems
- Support host-based countermeasures
  - e.g., address randomization, etc.
  - Make sure your implementation works on them
  - Microsoft EMET (provide info for it)
- Host-based sandboxing/wrappers
  - SELinux (provide starter policy)
  - Document inputs and outputs (files, ports)
- Network-based measures
  - Firewalls, intrusion detection/prevention systems, NATs
  - Don't assume client IP address you see == IP address client sees

When designing & implementing, prepare for security-related tools in the operational (fielded) setting

NIST SAMATE Tool Categories (partial)

- Assurance Case Tools
- **Safer Languages**
- Design/Modeling Verification Tools
- **Source Code Security Analyzers, Byte Code Scanners, Binary Code Scanners**
- **Web Application Vulnerability Scanners**
- Intrusion Detectors
- Network Scanners
- Requirements Verification Tools
- Architecture Design Tools
- **Dynamic Analysis Tools**
- Web Services Network Scanners
- Database Scanning Tools
- Anti-Spyware Tools
- Tool Integration Frameworks

Source: https://www.nist.gov/itl/ssd/software-quality-group/samate/samate-tool-survey

NAVSEA "Software Security Assessment Tools Review" (2009)

- Static analysis code scanning
- Source code fault injection
- Dynamic analysis
- Architectural analysis
- Pedigree analysis
- Binary code analysis
- Disassembler analysis
- Binary fault injection
- Fuzzing
- Malicious code detector
- Byte code analysis

Fool with a tool is still a fool (1)

- RealNetworks' RealPlayer/Helix Player vulnerabilities:
  - CVE-2005-0455 / iDEFENSE Security Advisory 03.01.05

    ```
    char tmp[256]; /* Flawfinder: ignore */
    ```

    strcpy(tmp, pScreenSize); /* Flawfinder: ignore */
  - CVE-2005-1766 / iDefense Security Advisory 06.23.05

    ```
    sprintf(pTmp,  /* Flawfinder: ignore */
    ```
  - CVE-2007-3410 / iDefense Security Advisory 06.26.07

    ```
    strncpy(buf, pos, len); /* Flawfinder:
    ignore */
    ```
  - Kudos to RealNetworks for revealing what happened!!
- Flawfinder: trivial static analysis tool
  - Lexical scanner for C code, reports vulnerability patterns
  - Comment "`Flawfinder: ignore`" disables next hit report

Fool with a tool is still a fool (2)

- Flawfinder correctly found the vulnerability!!
  - Someone then modified code, claiming not vulnerable
  - Yet these are obvious – not complex – vulnerabilities
  - Likely told "change code until no problems reported"
- Tools are ***useless*** unless you understand major types of vulnerabilities and how to fix them
  - Training on *tool* not the issue (this tool trivial to run)
  - Training on *developing secure programs* is critical
    - Must understand tools' purpose and what to do with results
    - e.g., must know what it means and what to do if tool says "*potential SQL injection vulnerability at line X*"

Adopting tool(s)

- Culture change required
  - More than just another tool
  - Tool will not solve anything in isolation
- Define objectives
- Train before use
  - Esp. software security - types of vulnerabilities, how to fix them
- Start with pilot – small and friendly group
- Start by focusing on relevant, easily-understood
  - Disable detection of most problems at beginning
- Appoint “champion” to advocate
- Later, build on success

Sources: *Chess, West, Chou, Ron Ritchey*